

AURA Forge Impact and Tool-Evolution Report

Baseline-vs-after metrics and controlled tool-evolution drill

All measurements apply only to three representative local capstone scenarios. No company-wide, production-wide, or future-scenario savings are inferred.

1. Baseline vs AURA Scenario Metrics

Scenario	Quality	Cost per successful run	Model roles	Human checkpoints
LOW	14/16 -> 16/16	\$0.6066006 -> \$0.3377550	5 -> 2	2 -> 0
MEDIUM	15/16 -> 16/16	\$0.9093231 -> \$0.7300815	5 -> 3	2 -> 1
HIGH	15/16 -> 16/16	\$1.1753241 -> \$1.1061042	5 -> 5	2 -> 2
Aggregate	44/48 -> 48/48	19.22% reduction	33.33% reduction	50% reduction

2. Required Impact Metrics

Metric requested	Evidence status	Reported value or proxy
Quality	Directly measured	LOW +2 points, MEDIUM +1, HIGH +1; aggregate 44/48 -> 48/48.
Review latency	Proxy only	Direct wall-clock review latency was not reliably measured. Human-checkpoint reduction is the documented proxy: 6 -> 3 aggregate.
Defect rate	Proxy only	No production defect rate exists. Quality-rubric pass/fail is the documented proxy: LOW control failed at 14/16; all AURA routes passed at 16/16.
Cycle time	Proxy only	Direct cycle time was not reliably measured. Route weight proxy: model roles 15 -> 10 and human checkpoints 6 -> 3.
Cost per run	Directly measured	Successful-route cost: \$2.6912478 -> \$2.1739407 aggregate.

3. Interpretation

- LOW shows the clearest gain because a documentation-only change was over-served by the fixed route.
- MEDIUM preserves engineering rigor through implementation, review, testing, and final approval while dropping unnecessary planning and project-manager overhead.
- HIGH deliberately keeps the full governed route; the small cost change is expected because sensitive work should not be optimized by removing controls.
- The adaptive-autonomy thesis is supported for the measured scenarios: use less autonomy where risk allows and keep governance where risk demands it.

4. Tool-Evolution Drill Summary

Question	Answer
Change attempted	Controlled commit 7c666e5 intentionally added continue-on-error: true to permanent policy-tests in .github/workflows/ci.yml.
What broke	The permanent policy gate became non-blocking, weakening the core governance guarantee.
What detected it	scripts/check-pipeline-integrity.py rejected the workflow with: policy-tests must not use continue-on-error.
What improved	The integrity checker proved it can distinguish optional advisory review from permanent gates.
What was fixed	Commit 4e446f6 removed continue-on-error: true from policy-tests and reran the integrity gate.
Final result	The repaired workflow passed integrity checks. The broken state was preserved as a controlled local drill and was never pushed as production work.

5. Boundary

The drill was intentional fault injection, not an accidental production incident and not a live deployment event. It demonstrates the self-improving gate behavior in a production-like capstone workflow only.

Source Evidence

Primary sources: docs/capstone/control-vs-aura-impact.md, docs/capstone/aura-results.md, docs/capstone/control-baseline-comparison.md, docs/capstone/tool-evolution-drill.md, docs/capstone/iteration-narrative.md.